# Fixing the Clock

private, accessors, constructors, and `this`

# Where we left off

Every instance variable was public, so `main()` could set `minutes` to 75, and nothing stopped it.

Whose job is it to keep a Clock sensible?

**The Clock's.** Today we give it the tools to do that job.

```
$ ./clock
09:75:00
09:76:00
```

```
72  int main() {
73      Clock myClock;
74
75      myClock.hours = 9;
76      myClock.minutes = 75;
77      myClock.print24();
78
79      for (int tick = 0; tick < 60; tick++) {
80          myClock.incrementSeconds();
81      }
82      myClock.print24();
83
84      return 0;
85  }
```

# `private`: members only

Anything after `private:` can only be used by the class's own member functions.

Anything after `public:` can be used by anyone, including `main()`.

The style guide puts instance variables at the top of the class, so `private:` comes first.

(note: some style guides put instance variables at the bottom instead of the top.)

```
6   class Clock {
7   private:
8       int hours = 0;
9       int minutes = 0;
10      int seconds = 0;
11
12  public:
13      void incrementSeconds();
14      void incrementMinutes();
15      void incrementHours();
16
17      void print24();
18      void print12();
19  };
```

# Now this won't compile

Good: `main()` can't break the Clock anymore. The compiler stops it before the program ever runs.

```
74  int main() {
75      Clock myClock;
76
77      myClock.minutes = 75;
78      myClock.print24();
79
80      return 0;
81  }
```

```
$ g++ clock.cpp
clock.cpp:77:13: error: 'int Clock::minutes' is private within this context
   77 |     myClock.minutes = 75;
      |             ^~~~~~
```

# …but neither will this

Now `main()` can't even *read* the time. We locked everyone out, including code with a good reason to look.

```
74  int main() {
75      Clock myClock;
76
77      for (int tick = 0; tick < 90; tick++) {
78          myClock.incrementSeconds();
79      }
80      cout << "minutes: " << myClock.minutes << endl;
81
82      return 0;
83  }
```

```
$ g++ clock.cpp
clock.cpp:80:36: error: 'int Clock::minutes' is private within this context
   80 |     cout << "minutes: " << myClock.minutes << endl;
      |                                    ^~~~~~
```

# Accessors: a read-only window

An **accessor** (or *getter*) is a public member function that hands back information.

`main()` can call `getMinutes()`, but it still can't change `minutes`. Reading: yes. Writing: no.

```
$ ./clock
minutes: 1
seconds: 30
```

```
25  int Clock::getHours() {
26      return hours;
27  }
28
29  int Clock::getMinutes() {
30      return minutes;
31  }
32
33  int Clock::getSeconds() {
34      return seconds;
35  }
```

# Interface

The 12-hour math from `print12()` now lives in `getHours12()` and `isPM()`.

Code calling these functions don't need to know *how* the answer gets worked out, it just receives the answer.

`print12()` now calls these two functions.

```
42  int Clock::getHours12() {
43      int displayHour = getHours() % 12;
44      if (displayHour == 0) {
45          displayHour = 12;
46      }
47      return displayHour;
48  }
49
50  bool Clock::isPM() {
51      return getHours() >= 12;
52  }
```

# `setTime(9, 75, 0)`: now what?

A **mutator** (or *setter*) is a public member function that changes private data, so it can check the new value first. But what should it do when the value is bad?

### Ignore it

Keep the old time. Say nothing.

### Refuse and report

Keep the old time. Return `false` so the caller knows.

### Reset

Go to `00:00:00`.

### Clamp

Use the closest valid time: `09:59:00`.

### Wrap around

75 minutes is an hour and 15 minutes: `10:15:00`.

### Complain

Print an error message.

Who finds out that something went wrong? Which of these could hide a bug for a long time?

# Our choice: refuse and report

`setTime()` checks *everything* before changing *anything*.

If any part is bad, it returns `false` and the clock keeps its old time. Otherwise it sets the time and returns `true`.

```
54  bool Clock::setTime(int newHours, int newMinutes,
55          int newSeconds) {
56      if (newHours < 0 || newHours > 23 ||
57              newMinutes < 0 || newMinutes > 59 ||
58              newSeconds < 0 || newSeconds > 59) {
59          return false;
60      }
61
62      hours = newHours;
63      minutes = newMinutes;
64      seconds = newSeconds;
65      return true;
66  }
```

# Using `setTime()`

The caller can check the return value and decide what to do.

The bad time never gets in: the clock stays at midnight until the good call.

```
$ ./clock
Not a real time!
00:00:00
21:05:03
9:05:03 PM
```

```
116 int main() {
117     Clock myClock;
118
119     if (!myClock.setTime(9, 75, 0)) {
120         cout << "Not a real time!" << endl;
121     }
122     myClock.print24();
123
124     myClock.setTime(21, 5, 3);
125     myClock.print24();
126     myClock.print12();
127
128     return 0;
129 }
```

# Why not just call the parameters `hours`?

Inside `setTime()`, the parameter `hours` *hides* the instance variable `hours`. So `hours = hours;` copies the parameter onto itself.

It compiles cleanly, with no warnings at all. And the clock never changes.

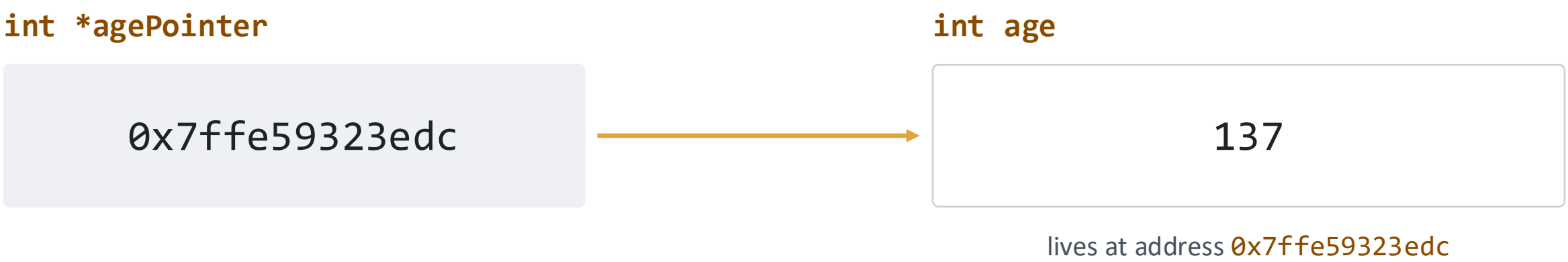
```
$ ./clock
00:00:00
```

```
54  bool Clock::setTime(int hours, int minutes,
55          int seconds) {
56      if (hours < 0 || hours > 23 ||
57              minutes < 0 || minutes > 59 ||
58              seconds < 0 || seconds > 59) {
59          return false;
60      }
61
62      hours = hours;
63      minutes = minutes;
64      seconds = seconds;
65      return true;
66  }
```

# A quick detour: what is a pointer?

Every variable lives somewhere in memory, and every spot in memory has an **address**: a number, usually written in hexadecimal.

A **pointer** is a variable that holds an address. It *points to* another variable.




`agePointer` doesn't hold 137. It holds *where to find* 137.

# Pointers in code

`int *agePointer` declares a pointer to an `int`.

`&age` means *the address of* `age`.

`*agePointer` means *the thing it points to*. Change that, and you change `age`.

```
$ ./pointer
age: 137
&age: 0x7ffe59323edc
agePointer: 0x7ffe59323edc
*agePointer: 137
age: 42
```

```
5   int main() {
6       int age = 137;
7       int *agePointer = &age;
8
9       cout << "age: " << age << endl;
10      cout << "&age: " << &age << endl;
11      cout << "agePointer: " << agePointer << endl;
12      cout << "*agePointer: " << *agePointer << endl;
13
14      *agePointer = 42;
15      cout << "age: " << age << endl;
16
17      return 0;
18  }
```

# `this`: a pointer to the object

Every member function gets a hidden pointer named `this`. It holds the address of the object the function was called on.

```
myClock.setTime(21, 5, 3);
```



`this` points at `myClock`

*inside setTime(), this time:*

| | | |
|---|---|---|
| **`this`** | means | the address of myClock |
| **`this->hours`** | means | myClock.hours (the instance variable) |
| **`hours`** | means | the parameter, 21 |

`->` follows a pointer to one of the object's members. `this->hours` is short for `(*this).hours`.

# Fixing `setTime()` with `this->`

`this->hours` is the instance variable. Plain `hours` is the parameter.

Two fixes, both fine: rename the parameters (`newHours`), or use `this->`. Pick one style and stick with it.

Many C++ codebases avoid the clash entirely by naming instance variables differently, like `hours_` or `m_hours`.

```
$ ./clock
21:05:03
```

```
54  bool Clock::setTime(int hours, int minutes,
55          int seconds) {
56      if (hours < 0 || hours > 23 ||
57              minutes < 0 || minutes > 59 ||
58              seconds < 0 || seconds > 59) {
59          return false;
60      }
61
62      this->hours = hours;
63      this->minutes = minutes;
64      this->seconds = seconds;
65      return true;
66  }
```

# Constructors

setting up an object the moment it's created

# Your first constructor

A **constructor** runs automatically when an object is created.

It has the same name as the class and *no return type*, not even `void`.

This one passes its job along to `setTime()`, so the checking only lives in one place.

```
33  Clock::Clock() {
34      // instance variables already start at 0
35  }
36
37  Clock::Clock(int hours, int minutes,
38          int seconds) {
39      setTime(hours, minutes, seconds);
40  }
```

# Using constructors

`Clock meeting(9, 30, 0);` creates the clock *and* sets its time in one line, like `int x = 137;`.

`Clock midnight;` uses the constructor with no parameters: the **default constructor**.

```
$ ./clock
09:30:00
00:00:00
```

```
128 int main() {
129     Clock meeting(9, 30, 0);
130     Clock midnight;
131 
132     meeting.print24();
133     midnight.print24();
134 
135     return 0;
136 }
```

# Where did `Clock midnight;` go?

If you write no constructors, C++ quietly gives you a default one. Write *any* constructor and that stops, so declare `Clock();` yourself if you still want `Clock midnight;` to work.

```
123 int main() {
124     Clock meeting(9, 30, 0);
125     Clock midnight;
126
127     meeting.print24();
128     midnight.print24();
129
130     return 0;
131 }
```

```
$ g++ clock.cpp
clock.cpp:125:11: error: no matching function for call to 'Clock::Clock()'
  125 |     Clock midnight;
      |           ^~~~~~~~
```

# What about `Clock broken(25, 0, 0);`?

A constructor can't return `false`. Ours calls `setTime()`, which refuses, so the clock is left at midnight.

Is that OK? You asked for 25:00:00 and silently got 00:00:00. What else could a constructor do?

```
$ ./clock
00:00:00
```

```
128 int main() {
129     Clock broken(25, 0, 0);
130
131     broken.print24();
132
133     return 0;
134 }
```

# Why bother?

hiding how a Clock works

# What if a Clock stored the time differently?

Inside, a Clock could keep just one number: `int secondsSinceMidnight = 0;` The accessors do a little math to hand back hours, minutes, and seconds.

Everything outside the class still calls `getHours()`, exactly like before.

```
44  int Clock::getHours() {
45      return secondsSinceMidnight / SECONDS_PER_HOUR;
46  }
47
48  int Clock::getMinutes() {
49      return secondsSinceMidnight / SECONDS_PER_MINUTE % 60;
50  }
51
52  int Clock::getSeconds() {
53      return secondsSinceMidnight % SECONDS_PER_MINUTE;
54  }
```

# Rollover becomes arithmetic

The whole chain of `if` statements turns into one line each. `%` wraps the count back to 0 at midnight.

```
81  void Clock::incrementSeconds() {
82      secondsSinceMidnight = (secondsSinceMidnight + 1) % SECONDS_PER_DAY;
83  }
84
85  void Clock::incrementMinutes() {
86      secondsSinceMidnight = (secondsSinceMidnight + SECONDS_PER_MINUTE) % SECONDS_PER_DAY;
87  }
88
89  void Clock::incrementHours() {
90      secondsSinceMidnight = (secondsSinceMidnight + SECONDS_PER_HOUR) % SECONDS_PER_DAY;
91  }
```

# `main()` didn't change. Not one line.

Same `main()`, same output, completely different insides.

That's why we make things private: code outside the class only knows *what* a Clock can do, not *how*. So the how is free to change.

*hours, minutes, seconds*

```
$ ./clock_hms
09:31:30
9:31:30 AM
00:00:00
```

*secondsSinceMidnight*

```
$ ./clock_secs
09:31:30
9:31:30 AM
00:00:00
```

```
115 int main() {
116     Clock meeting(9, 30, 0);
117     for (int tick = 0; tick < 90; tick++) {
118         meeting.incrementSeconds();
119     }
120     meeting.print24();
121     meeting.print12();
122
123     Clock lateNight(23, 59, 59);
124     lateNight.incrementSeconds();
125     lateNight.print24();
126
127     return 0;
128 }
```

# Today's vocabulary

### private

Only the class's own member functions can use it.

### accessor (getter)

A public member function that hands back information.

### mutator (setter)

A public member function that changes private data, after checking it.

### constructor

Runs automatically when an object is created. Same name as the class, no return type.

### default constructor

The constructor with no parameters: `Clock midnight;`

### this

A pointer to the object a member function was called on.

# Next time

Pointers: what `this` really is, and what else pointers can do